

Random f

Game Theory

```
int k;cin >> k;
int s[110];
int g[10000+10] = {};
for(int i = 0;i<k;i++)cin >> s[i];
for(int i = 1;i<=10000;i++){
    set<int>reach;
    for(int j = 0;j<k;j++){
        if(i-s[j] >= 0){
            reach.insert(g[i-s[j]]);
        }
    }
    for(auto ii : reach){
        if(g[i] == ii)g[i]++;
        else break;
    }
}

int m;cin >> m;
for(int i = 0;i<m;i++){
    int l;cin >> l;
    int ans = 0;
    for(int j = 0;j<l;j++){
        int x;cin >> x;
        ans ^= g[x];
    }
    if(!ans)cout << 'L';else cout << 'W';
}
```

Matrix

```
const int mod = 1e9 + 7;

int add(ll a, int b) {
    return (a + b) % mod;
}
```

```

int mul(ll a, int b) {
    return (a * b) % mod;
}

struct Matrix {
    vector<vector<int>> mat;
    int n, m;

    Matrix(int _n, int _m, int val = 0) {
        n = _n, m = _m;
        mat = vector<vector<int>>(n, vector<int>(m, val));
    }

    Matrix operator*(const Matrix &other) const {
        int n = mat.size();
        int m = other.mat[0].size();
        int k = mat[0].size();
        assert(mat[0].size() == other.mat.size());
        Matrix ret(n, m);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++)
                for (int l = 0; l < k; l++)
                    ret.mat[i][j] = add(ret.mat[i][j], mul(mat[i][l],
other.mat[l][j]));
        return ret;
    }
};

Matrix I(int n) {
    Matrix ret(n, n);
    for (int i = 0; i < n; i++) ret.mat[i][i] = 1;
    return ret;
}

Matrix mult(Matrix &a, Matrix &b) {
    int n = a.mat.size();
    int m = b.mat[0].size();
    int k = a.mat[0].size();
    assert(a.mat[0].size() == b.mat.size());
    Matrix ret(n, m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            for (int l = 0; l < k; l++)
                ret.mat[i][j] = add(ret.mat[i][j], mul(a.mat[i][l], b.mat[l]
[j]));
}

```

```

        return ret;
    }

    Matrix fast_power(Matrix base, int power) {
        Matrix ret = I(base.n);
        while (power > 0) {
            if (power & 1) ret = ret * base;
            base = base * base;
            power >>= 1;
        }
        return ret;
    }
}

```

Random

Custom Sort

```

Priority queue custom sort
class Compare
{
public:
    bool operator()(int below, int above)
    {
        return below < above;
    }
};

priority_queue<int, vector<int>, Compare> pq;
Set & multiset custom sort
struct Compare
{
    bool operator()(const int& x, const int& y) const
    {
        return x < y;
    }
};

set<int, Compare> st;
multiset<int, Compare> ms;
Ordered set & multiset
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template<typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;

```

```

template<typename T>
using ordered_multiset = tree<T, null_type, less_equal<T>, rb_tree_tag,
tree_order_statistics_node_update>;

template<class T> struct Multiset {

    ordered_multiset<T> ms;

    void insert(const T& x) {
        ms.insert(x);
    }
    bool exist(const T& x) const {
        auto it = ms.upper_bound(x);
        if (it == ms.end()) return false;
        return *it == x;
    }
    bool erase(const T& x) {
        if (!exist(x)) return false;
        ms.erase(ms.upper_bound(x));
        return true;
    }
    T operator [] (int p) const {
        assert(p >= 0 && p < (int)ms.size());
        return *ms.find_by_order(p);
    }
    typename ordered_multiset<T>::iterator begin() { return ms.begin(); }
    typename ordered_multiset<T>::iterator end() { return ms.end(); }
    int first(const T& x) const {
        if (!exist(x)) return -1;
        return ms.order_of_key(x);
    }
    int last(const T& x) const {
        if (!exist(x)) return -1;
        if ((*this)[ms.size() - 1] == x) return ms.size() - 1;
        return first(*ms.lower_bound(x)) - 1;
    }
    int lower(const T& x) const { // returns the index
        if ((*this)[ms.size() - 1] < x) return -1;
        return ms.order_of_key(x);
    }
    int count(const T& x) const {
        if (!exist(x)) return 0;
        return last(x) - first(x) + 1;
    }
}

```

```

    int size() const { return ms.size(); }
    void clear() { ms.clear(); }
};

```

2D prefix and partial

```

//2D prefix sum
template<typename T = ll> struct pref_2D {
    int n, m;
    vector<vector<T>> pref;

    pref_2D(const vector<vector<T>>& x) {
        n = x.size();
        m = x[0].size();
        pref.resize(n, vector<T>(m, 0));

        for (int j = 0; j < m; j++) {
            pref[0][j] = x[0][j];
            for (int i = 1; i < n; i++)
                pref[i][j] = pref[i - 1][j] + x[i][j];
        }
        for (int i = 0; i < n; i++)
            for (int j = 1; j < m; j++)
                pref[i][j] += pref[i][j - 1];
    }

    ll get(int x, int y) {
        if (x < 0 || y < 0) return 0;
        return pref[x][y];
    }

    ll get(int i1, int j1, int i2, int j2) {
        T ret = get(i2, j2) - get(i2, j1 - 1);
        ret -= get(i1 - 1, j2) - get(i1 - 1, j1 - 1);
        return ret;
    }
};

//2D partial sum
struct partial_sum_2D{
    int n, m;
    vector<vector<ll>> upd;
    partial_sum_2D(int _n, int _m) {
        n = _n, m = _m;
        upd.resize(n, vector<ll>(m));
    }

    void update(int x1, int y1, int x2, int y2, ll val) {
        upd[x1][y1] += val;
    }
};

```

```

    if (x2 + 1 < n) upd[x2 + 1][y1] -= val;
    if (y2 + 1 < m) upd[x1][y2 + 1] -= val;
    if (x2 + 1 < n and y2 + 1 < m) upd[x2 + 1][y2 + 1] += val;
}
void calc() {
    for (int i = 1; i < n; i++)
        for (int j = 0; j < m; j++) upd[i][j] += upd[i - 1][j];

    for (int i = 0; i < n; i++)
        for (int j = 1; j < m; j++) upd[i][j] += upd[i][j - 1];
}
};

```

files

```

freopen("in.txt", "r", stdin);
freopen("out.txt", "w", stdout);

```

RNG Setup (C++)

Always use `mt19937` instead of `rand()`. It is faster, has a much larger period ($2^{13} - 1$), and produces higher-quality randomness.

C++

```

#include <bits/stdc++.h>
using namespace std;

// Seed with the current time to ensure different results every run
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

// Generate a random 32-bit integer
int x = rng();

// For 64-bit integers, use mt19937_64
mt19937_64 rng64(chrono::steady_clock::now().time_since_epoch().count());

```

2. Random Integer in Range `[, ]`

Using the modulo operator (`%`) can introduce **modulo bias**. The `uniform_int_distribution` is the mathematically correct way to handle ranges.

C++

```

int L = 1, R = 100;
uniform_int_distribution<int> dist(L, R);
int x = dist(rng);

// For long long:
uniform_int_distribution<long long> dist_ll(1LL, 1e18);
long long y = dist_ll(rng);

```

Mono tonic things

```

#include <bits/stdc++.h>
using namespace std;

struct MonotonicMinDeque {
    deque<int> dq;
    vector<long long> *arr;

    MonotonicMinDeque(vector<long long> &a) {
        arr = &a;
    }

    void push(int i) {
        while (!dq.empty() && (*arr)[dq.back()] >= (*arr)[i])
            dq.pop_back();
        dq.push_back(i);
    }

    void pop(int i) {
        if (!dq.empty() && dq.front() == i)
            dq.pop_front();
    }

    long long get() {
        return (*arr)[dq.front()];
    }

    bool empty() {
        return dq.empty();
    }
};

```

```

// Function to implement monotonic increasing stack
vector<int> monotonicIncreasing(vector<int>& nums)
{

```

```

int n = nums.size();
stack<int> st;
vector<int> result;

// Traverse the array
for (int i = 0; i < n; ++i) {

    // While stack is not empty AND top of stack is more
    // than the current element
    while (!st.empty() && st.top() > nums[i]) {

        // Pop the top element from the
        // stack
        st.pop();
    }

    // Push the current element into the stack
    st.push(nums[i]);
}

// Construct the result array from the stack
while (!st.empty()) {
    result.insert(result.begin(), st.top());
    st.pop();
}

return result;
}

vector<int> nextGreater(vector<int> &a) {
    int n = a.size();
    vector<int> nxt(n, -1);
    stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() and a[i] > a[st.top()]) {
            nxt[st.top()] = i;
            st.pop();
        }
        st.push(i);
    }
    return nxt;
}

vector<int> prevGreater(vector<int> &a) {
    int n = a.size();
    vector<int> pre(n, -1);

```

```
stack<int> st;
for (int i = 0; i < n; i++) {
    while (!st.empty() && a[st.top()] <= a[i])
        st.pop();
    if (!st.empty()) pre[i] = st.top();
    st.push(i);
}
return pre;
}
```